

Task 4 Report

Stanisław Apanasiewicz 198169

Subtask 1

Use case 1 – NASA decides to change star's constellation it belongs to. The database has to record:

- Replacing a star's previous constellation association to a new constellation name (a change in a foreign key Star.constellation_name)
- Subtracting the number of stars in an old constellation. A change in the record Star_constellation.no_stars (number of stars) by one ($\text{no_stars} = \text{no_stars} - 1$)
- Incrementing the number of stars in a new constellation. A change in the record Star_constellation.no_stars (number of stars) by one ($\text{no_stars} = \text{no_stars} + 1$)

Use case 2 – An observer decides to end working in an observatory. The database has to

- Set the date of the end of a work to the current date (Work.date_of_work_end = [so-me-date])
- If the work done by the boss of an observatory, the record Observatory.boss_id has to be set to NULL or switched.

Use case 3 (I decided to add a new records)

- Budget of an observatory in dollars (Observatory.budget)
- Cost of monitoring an object for a year in dollars (Monitor_schedule.cost)

Each year an observatory has to pay from its budget for the costs of monitoring.
The database has to:

- Check whether the balance is positive
- Subtract a budget of an observatory by a sum of costs of every celestial body it monitors.

Subtask 2

Use case 1 – Read committed, it is sufficient here, because the incrementation of the number of stars is very fast and does not require reading a value from another entity, just (SET no_stars = no_stars + 1). The query also does not display anything

Use case 2 - Read committed, it is sufficient here, because the query only sets the date from NULL to a selected and sometimes (very rarely) it sets the boss record to NULL. There are no reasonable scenarios where the record Work.date_of_work_end would be changed by another transaction so fuzzy read is not scary.

Use case 3 REPEATABLE READ –we don't need serializable here even though the account of an observatory is debited by a sum of all costs of the monitoring celestial bodies. If someone added a new body there could occur a phantom, but the table reads the cost of the monitoring only once. We need however the Repeatable Read, because we check if the account balance is positive to even start the transaction. Only then we debit it, in this meantime there can be another transaction which debits the transaction so we have to avoid a fuzzy read.

Added queries:

altering.sql

```
1  ALTER TABLE Observatory --add a budget to an observatory (how much money does it have)
2      ADD budget DECIMAL(15, 2);
3  ALTER TABLE Monitor_schedule --add a yearly cost for monitoring a planet
4      ADD cost DECIMAL(15, 2);
5
6
7  UPDATE Observatory SET budget = 15000000.00 WHERE observatory_id = 'G1000001';
8  UPDATE Observatory SET budget = 12500000.00 WHERE observatory_id = 'G1000002';
9  UPDATE Observatory SET budget = 9000000.00 WHERE observatory_id = 'G1000003';
10 UPDATE Observatory SET budget = 2000000.00 WHERE observatory_id = 'G1000004';
11 UPDATE Observatory SET budget = 5500000.00 WHERE observatory_id = 'G1000005';
12 UPDATE Observatory SET budget = 8000000.00 WHERE observatory_id = 'G1000006';
13 UPDATE Observatory SET budget = 3000000.00 WHERE observatory_id = 'G1000007';
14 UPDATE Observatory SET budget = 4200000.00 WHERE observatory_id = 'G1000008';
15 UPDATE Observatory SET budget = 11000000.00 WHERE observatory_id = 'G1000009';
16 UPDATE Observatory SET budget = 6700000.00 WHERE observatory_id = 'G1000010';
17
18 UPDATE Observatory SET budget = 90000000.00 WHERE observatory_id = 'S2000001';
19 UPDATE Observatory SET budget = 800000000.00 WHERE observatory_id = 'S2000002';
20 UPDATE Observatory SET budget = 65000000.00 WHERE observatory_id = 'S2000003';
21 UPDATE Observatory SET budget = 72000000.00 WHERE observatory_id = 'S2000004';
22 UPDATE Observatory SET budget = 55000000.00 WHERE observatory_id = 'S2000005';
23 UPDATE Observatory SET budget = 45000000.00 WHERE observatory_id = 'S2000006';
24 UPDATE Observatory SET budget = 10000000.00 WHERE observatory_id = 'S2000007';
25 UPDATE Observatory SET budget = 10000000.00 WHERE observatory_id = 'S2000008';
26 UPDATE Observatory SET budget = 85000000.00 WHERE observatory_id = 'S2000009';
27 UPDATE Observatory SET budget = 50000000.00 WHERE observatory_id = 'S2000010';
28
29 UPDATE Monitor_Schedule SET cost = 5000.00 WHERE monitor_id = 10000000001;
30 UPDATE Monitor_Schedule SET cost = 12000.00 WHERE monitor_id = 10000000002;
31 UPDATE Monitor_Schedule SET cost = 4500.00 WHERE monitor_id = 10000000003;
32 UPDATE Monitor_Schedule SET cost = 8000.00 WHERE monitor_id = 10000000004;
33 UPDATE Monitor_Schedule SET cost = 15000.00 WHERE monitor_id = 10000000005;
34 UPDATE Monitor_Schedule SET cost = 3000.00 WHERE monitor_id = 10000000006;
35 UPDATE Monitor_Schedule SET cost = 2500.00 WHERE monitor_id = 10000000007;
36 UPDATE Monitor_Schedule SET cost = 11000.00 WHERE monitor_id = 10000000008;
37 UPDATE Monitor_Schedule SET cost = 9500.00 WHERE monitor_id = 10000000009;
38 UPDATE Monitor_Schedule SET cost = 13500.00 WHERE monitor_id = 10000000010;
39 UPDATE Monitor_Schedule SET cost = 2000.00 WHERE monitor_id = 10000000011;
40 UPDATE Monitor_Schedule SET cost = 2200.00 WHERE monitor_id = 10000000012;
41 UPDATE Monitor_Schedule SET cost = 7500.00 WHERE monitor_id = 10000000013;
42 UPDATE Monitor_Schedule SET cost = 6000.00 WHERE monitor_id = 10000000014;
43 UPDATE Monitor_Schedule SET cost = 8500.00 WHERE monitor_id = 10000000015;
```

Transaction1.sql (with check)

```
1  --planet Vega moves from constellation Lyra to constellation Dorado
2  SELECT *
3  FROM Star_Constellation
4  WHERE constellation_name IN ('Lyra', 'Dorado')
5
6  GO
7
8  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
9  BEGIN TRANSACTION;
10
11  UPDATE Star
12  SET Star.constellation_name = 'Dorado'
13  WHERE star_name = 'Vega';
14
15  UPDATE star_constellation
16  SET no_stars = no_stars - 1
17  WHERE constellation_name = 'Lyra';
18
19  UPDATE star_constellation
20  SET no_stars = no_stars + 1
21  WHERE constellation_name = 'Dorado';
22
23  COMMIT TRANSACTION;
24
25  GO
26
27  SELECT *
28  FROM Star_Constellation
29  WHERE constellation_name IN ('Lyra', 'Dorado')
30
```

77 % Nie znaleziono żadnych problemów W.: 18, Zn.: 39 SPACJE CRLF Windows 1250

Wyniki Komunikaty

	constellation_name	no_stars
1	Dorado	31
2	Lyra	59

	constellation_name	no_stars
1	Dorado	32
2	Lyra	58

✓ Zapytanie zostało wykonane pomyślnie. sql198169server.database.wi... s198169 (53) s198169-space 00:00:00 Wiersz: 1, kolumna: 1 Wiersze: 4

Transaction2.sql (with check)

```
1  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
2  -- Agata Gwiezdna 'F1234567890' leaves from Mauna Kea 'G1000001', she was not the boss
3  BEGIN TRANSACTION;
4
5  UPDATE Work
6  SET date_of_end = '2026-01-26'
7  WHERE observer_id = 'F1234567890'
8  AND observatory_id = 'G1000001'
9  AND date_of_end IS NULL;
10
11  UPDATE Observatory
12  SET boss_id = NULL
13  WHERE boss_id = 'F1234567890'
14  AND Observatory_id = 'G1000001';
15
16  COMMIT TRANSACTION;
17
18  SELECT observer_name, observer_surname, date_of_start, date_of_end FROM Work
19  INNER JOIN Observer
20  ON Work.observer_id = Observer.observer_id
21  WHERE Work.observer_id = 'F1234567890'
```

93 % Nie znaleziono żadnych problemów W.: 13, Zn.: 34 SPACJE LF Windows 1250

Wyniki Komunikaty

	observer_name	observer_surname	date_of_start	date_of_end
1	Agata	Gwiezdna	2015-06-01	2026-01-26

✓ Zapytanie zostało wykonane pomyślnie. sql198169server.database.wi... s198169 (54) s198169-space 00:00:00 Wiersz: 1, kolumna: 1 Wiersze: 1

Transaction3v2.sql Part1

```
1  --SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
2  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
3  --SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
4
5  BEGIN TRANSACTION;
6
7  DECLARE @InitialBudget DECIMAL(15, 2);
8
9  SELECT @InitialBudget = budget
10 FROM Observatory
11 WHERE observatory_id = 'S2000001';
12
13 PRINT 'INITIAL BUDGET: ';
14 PRINT @InitialBudget;
15
16 IF @InitialBudget <= 0
17 BEGIN
18     PRINT 'OBSERVATORY IS BROKE';
19     ROLLBACK TRANSACTION;
20     RETURN;
21 END
22 --WAITFOR DELAY '00:00:10'; --execute the disruptor (phantom) transaction
23
24 DECLARE @TotalCost DECIMAL(15, 2); -- calculate the total cost of the monitoring
25
26 SELECT @TotalCost =
27     ISNULL(SUM(cost), 0.00)
28 FROM Monitor_Schedule MS
29 INNER JOIN Celestial_Body
30 ON MS.celestial_name = Celestial_Body.body_name
31 WHERE MS.observatory_id = 'S2000001'
32 GROUP BY MS.observatory_id;
33
34 PRINT 'TOTAL COST: ';
35 PRINT @TotalCost;
36
```

85 % Nie znaleziono żadnych problemów W.: 67, Zn.: 1 MIESZANE LF Windows 1250

Wyniki Komunikaty
Czas wykonania: 2026-01-24 17:07:20.4862077+01:00

85 % Nie znaleziono żadnych problemów W.: 15, Zn.: 1 TABULATORY MIESZANE UTF-8 with BOM

✓ Zapytanie zostało wykonane pomyślnie. sql198169server.database.wi... s198169 (88) s198169-space 00:00:10 Wiersz: 1, kolumna: 1 Wiersze: 1

```
34 PRINT 'TOTAL COST: ';
35 PRINT @TotalCost;
36
37 WAITFOR DELAY '00:00:10'; --execute the disruptor (fuzzy read) transaction
38
39 IF @InitialBudget < @TotalCost
40 BEGIN
41     PRINT 'OBSERVATORY CANNOT AFFORD MONITOR PAYMENTS';
42     ROLLBACK TRANSACTION;
43     RETURN;
44 END
45 ELSE
46 BEGIN
47     UPDATE Observatory
48     SET budget = budget - @TotalCost
49     WHERE observatory_id = 'S2000001';
50
51 DECLARE @FinalBudget DECIMAL(15, 2);
52
53 SELECT @FinalBudget = budget
54 FROM Observatory
55 WHERE observatory_id = 'S2000001';
56
57 PRINT 'INITIAL BUDGET - TOTAL COST ';
58 PRINT (@InitialBudget - @TotalCost);
59
60 PRINT 'FINAL BUDGET (it should be the same as above): ';
61 PRINT @FinalBudget;
62
63 COMMIT TRANSACTION;
64
65 SELECT budget
66 FROM Observatory
67 WHERE observatory_id = 'S2000001';
68
```

85 % Nie znaleziono żadnych problemów W.: 67, Zn.: 1 MIESZANE LF Windows 1250

Wyniki Komunikaty
Czas wykonania: 2026-01-24 17:07:20.4862077+01:00

85 % Nie znaleziono żadnych problemów W.: 15, Zn.: 1 TABULATORY MIESZANE UTF-8 with BOM

✓ Zapytanie zostało wykonane pomyślnie. sql198169server.database.wi... s198169 (88) s198169-space 00:00:10 Wiersz: 1, kolumna: 1 Wiersze: 1

Check_for_T3.sql (query for visualising how much an account will be debited)

```

1 CREATE OR ALTER VIEW Observatory_spending AS
2
3 SELECT MS.observatory_id, MAX(observatory_name) AS observatory_name,
4        COUNT(MS.celestial_name) AS no_observatories_monitoring,
5        SUM(cost) AS Total_cost_per_year
6 FROM Monitor_Schedule MS
7     INNER JOIN Celestial_Body
8     ON MS.celestial_name = Celestial_Body.body_name
9     INNER JOIN Observatory
10    ON MS.observatory_id = Observatory.observatory_id
11 GROUP BY MS.observatory_id;
12
13 GO
14 -- calculate the sum of the spendings on monitoring made by a particular observatory
15
16 SELECT * FROM Observatory_spending
17 WHERE observatory_id = 'S2000001'
18
19

```

77 % Nie znaleziono żadnych problemów W.: 13, Zn.: 3 TABULATORY CRLF Windows 1250

Wyniki Komunikaty

observatory_id	observatory_name	no_observatories_monitoring	Total_cost_per_year
1 S2000001	Hubble Space Telescope	4	37500.00

Zapytanie zostało wykonane pomyślnie. sql198169server.database.wi... s198169 (81) s198169-space 00:00:00 Wiersz: 1, kolumna: 1 Wiersze: 1

T4.sql (a transaction to see what happens if a budget is debited by another transaction)

```

1 SELECT budget
2 FROM Observatory
3 WHERE observatory_id = 'S2000001';
4
5 GO
6
7 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED; -- a disruptor transaction
8 -- just to see what happens
9 BEGIN TRANSACTION;
10
11 UPDATE Observatory
12 SET budget = budget / 2
13 WHERE observatory_id = 'S2000001';
14
15 COMMIT TRANSACTION;
16
17 SELECT budget
18 FROM Observatory
19 WHERE observatory_id = 'S2000001';

```

I also have files PHANTOM.sql:

```

INSERT INTO Monitor_Schedule (monitor_id, celestial_name, observatory_id, date_of_start, date_of_end, cost) VALUES
(100000000067, 'WASP-12b', 'S2000001', '01-01', '01-05', 90000001.00 )

```

And DropPhantom.sql

```

DELETE FROM Monitor_Schedule
WHERE monitor_id = 100000000067;

```

But they show that we don't need serializable

